

MOVIE RECOMMENDATION SYSTEM
PROJECT DOCUMENTATION
INTERNSHIP TASK 03 – AI / ML

Name: Shazmeen

Program: AI / ML Internship

1. Introduction:

This project is a simple movie recommendation system developed for the Week 03 AI/ML internship task. The main purpose of this project is to recommend movies that are similar to a movie selected by the user. For this project, I used the MovieLens dataset. I worked mainly with movie titles and genres to find similar movies. The data was first cleaned and prepared before building the recommendation system. I used TF-IDF to convert movie genres into numerical features and cosine similarity to compare the movies. A simple Gradio interface was also created so that a user can enter a movie title and get similar movie recommendations.

2. Project Objective:

The main objective of this project is to build a recommendation system that can suggest movies based on the similarity of their genres.

The system should:

- Use a suitable movie dataset.
- Clean and prepare the dataset.
- Process movie genres.
- Find similarity between movies.
- Generate movie recommendations.
- Evaluate the recommendations using Precision@5.
- Provide a simple interface for users.

3. Dataset:

For this project, I used the **MovieLens Latest Small dataset**. The dataset was downloaded from the official GroupLens MovieLens dataset source and used in Google Colab.

The dataset contains different files, but for this project I mainly used:

- `movies.csv`
- `ratings.csv`

The `movies.csv` file contains information about movie IDs, movie titles, and genres.

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

The `ratings.csv` file contains user IDs, movie IDs, ratings, and timestamps.

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

I used the movie information for the main recommendation process and used the ratings data to prepare user-item interaction data.

4. Dataset Description:

The movie dataset contains the following important columns:

- **movieId:**
A unique ID assigned to each movie.
- **title:**
The name of the movie.
- **genres:**
The genres associated with the movie, such as Action, Comedy, Drama, Animation, etc.

The ratings dataset contains:

- **userId:**
The ID of the user who gave the rating.
- **movieId:**
The ID of the movie that was rated.
- **rating:**
The rating given by the user.
- **timestamp:**
The time when the rating was given.

5. Data Preparation:

Before creating the recommendation system, I checked the dataset to understand its structure and quality.

I checked:

- First few records of the dataset.
- Dataset information.
- Missing values.
- Duplicate records.
- Relevant columns.

For the movie data, I removed records where the movie title or genre information was missing. I also removed duplicate records so that the same movie information was not unnecessarily repeated.

After that, I kept only the columns needed for the recommendation system:

- `movieId`
- `title`
- `genres`

```
... Missing values in movies dataset:
movieId    0
title      0
genres     0
dtype: int64

Missing values in ratings dataset:
userId     0
movieId    0
rating     0
timestamp  0
dtype: int64
```

```
Duplicate rows in movies dataset: 0
Duplicate rows in ratings dataset: 0
```

6. Preparing Movie Genres:

The genres in the dataset were originally separated using the | symbol.

For example:

```
Adventure|Animation|Children|Comedy
```

I replaced the | symbol with spaces so that the genre information could be processed more easily by TF-IDF.

After this step, the genre information became suitable for feature extraction.

7. User-Item Interaction Data:

I also prepared the user-item interaction data from the ratings dataset.

For this, I selected:

- userId
- movieId
- rating

I then created a user-item interaction matrix using the ratings. In this matrix, users are represented as rows and movies are represented as columns. The ratings show the interaction between a user and a movie. Although the final recommendation method in this project is content-based filtering, this step was included as part of the data preparation required for recommendation-system development.

8. Recommendation Approach:

For this project, I selected **Content-Based Filtering**. I selected this approach because the project focuses on recommending movies similar to a movie selected by the user. The system looks at the genres of the selected movie and compares them with the genres of other movies. For example, if a user selects a movie that belongs to Animation and Comedy, the system looks for other movies with similar genre information. This allows the system to generate recommendations based on the characteristics of the selected movie.

9. Feature Extraction Using TF-IDF:

To compare the movie genres, I used **TF-IDF (Term Frequency-Inverse Document Frequency)**. TF-IDF converts the text information into numerical values. In this project, the movie genres were treated as text features. The TF-IDF values were then used to represent each movie in numerical form. This made it possible to calculate the similarity between different movies.

10. Calculating Cosine Similarity:

After converting the genres into numerical features, I used **cosine similarity**. Cosine similarity measures how similar two feature vectors are. In this project, it is used to compare the genre information of movies. The similarity values are then sorted from highest to lowest. The movies with the highest similarity are selected as recommendations.

11. Recommendation Engine:

A recommendation function was created to generate the results. The user provides a movie title to the function. The system first checks whether that movie exists in the dataset. If the movie is found, its similarity scores with other movies are calculated. The movies are then sorted according to their similarity scores, and the top recommended movies are returned. For example, when I tested the system with **Toy Story (1995)**, it generated recommendations such as:

- Antz (1998)
- Toy Story 2 (1999)
- Adventures of Rocky and Bullwinkle, The (2000)

- Emperor's New Groove, The (2000)
- Monsters, Inc. (2001)

*** Selected Movie: Toy Story (1995)

	Movie	Similarity Score
0	Antz (1998)	1.0
1	Toy Story 2 (1999)	1.0
2	Adventures of Rocky and Bullwinkle, The (2000)	1.0
3	Emperor's New Groove, The (2000)	1.0
4	Monsters, Inc. (2001)	1.0

12. Recommendation Scores:

I also added similarity scores to the recommendation results. These scores show how similar the recommended movie is to the movie selected by the user according to the genre features used by the system. For the Toy Story example, the recommended movies received a similarity score of **1.0** under the genre-based features used in this project. This means that the selected genre representation had maximum cosine similarity according to the features used. It does not mean that the movies are exactly the same in every aspect.

13. Evaluation Method:

To evaluate the recommendation system, I used **Precision@5**. Precision@5 checks the first five recommendations and calculates how many of them meet the relevance condition used for this project. For this project, a recommendation was considered relevant when it had at least one genre in common with the selected movie.

The formula used was:

$$\text{Precision@5} = \text{Relevant Recommendations} / 5$$

I tested the recommendation system on multiple movies instead of checking only one movie.

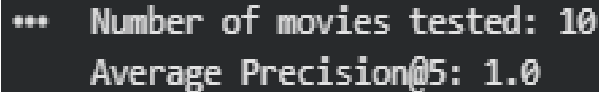
14. Evaluation Results:

The system was tested on **10 movies**.

The average Precision@5 obtained was:

Average Precision@5 = 1.00

This means that, based on the relevance rule defined for this project, all of the top-five recommendations in the tested cases had at least one genre in common with the selected movie. The result should be understood specifically according to the evaluation method used here. It does not mean that the recommendation system will always be perfect for every user or every type of movie.

A terminal window with a dark background and light green text. It displays two lines of output: '*** Number of movies tested: 10' and 'Average Precision@5: 1.0'.

```
*** Number of movies tested: 10
Average Precision@5: 1.0
```

15. Demonstration Interface:

For the demonstration part, I created a simple interface using **Gradio**.

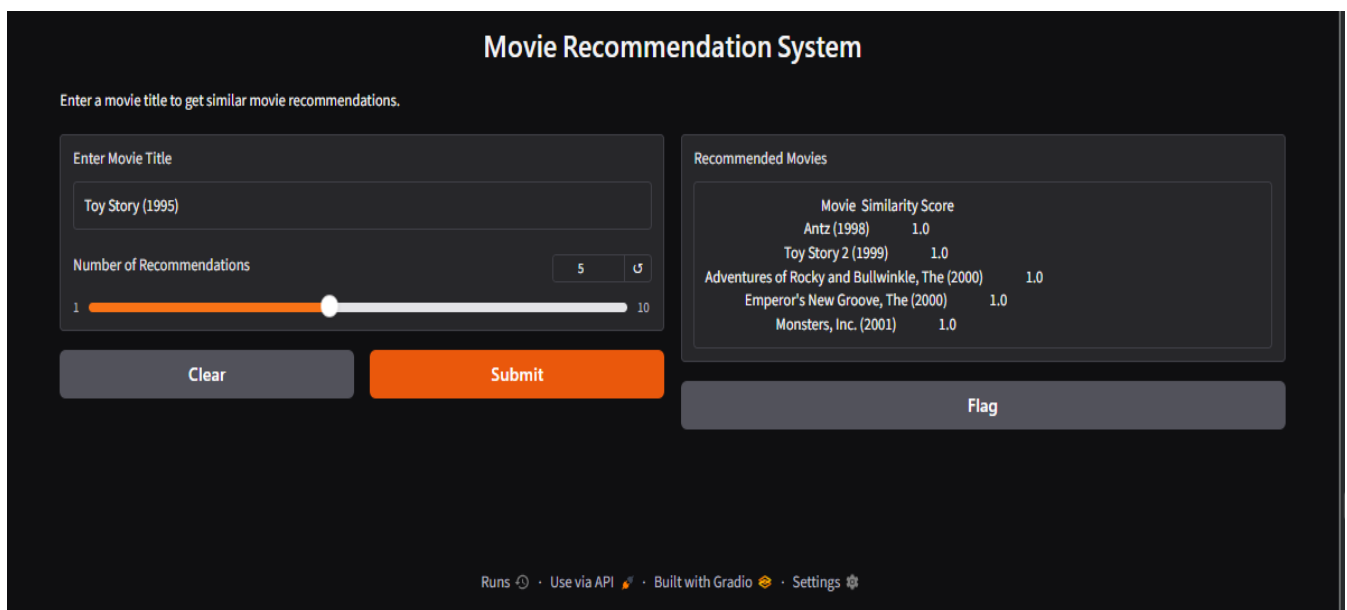
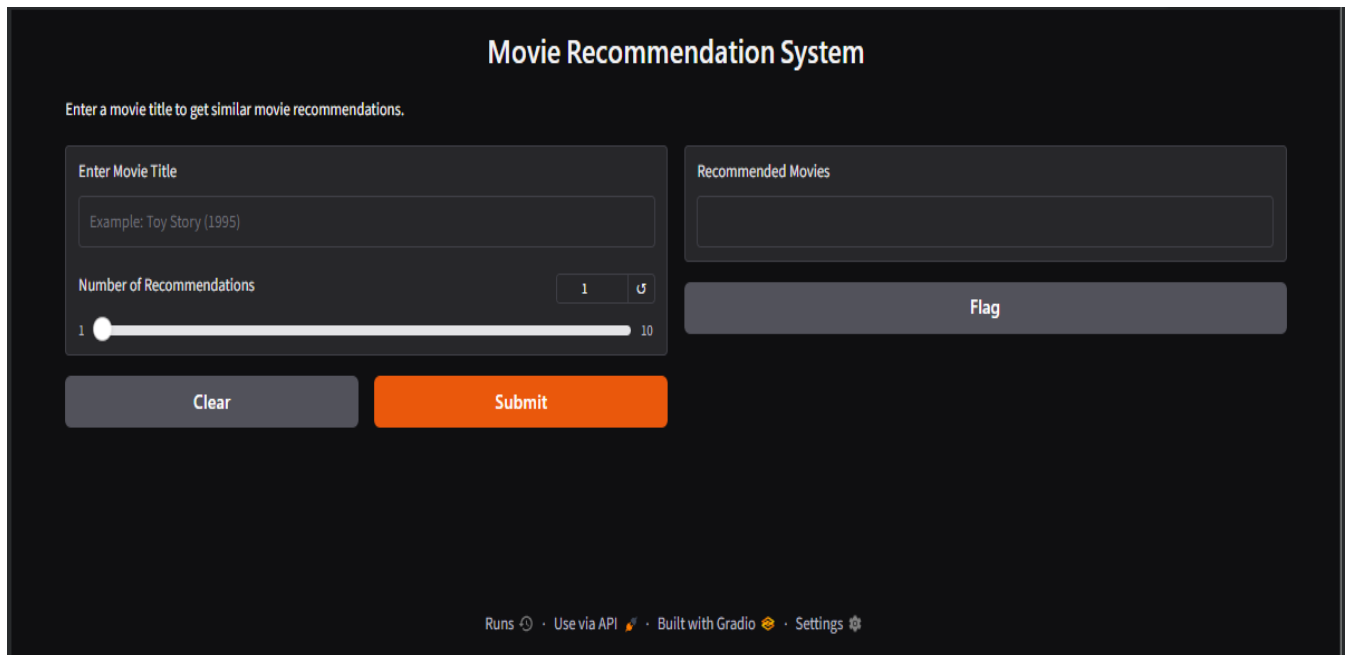
The interface allows the user to:

- Enter a movie title.
- Select the number of recommendations.
- View the recommended movies.
- See the similarity scores.

For example, the user can enter:

Toy Story (1995)

and select the number of movies they want to receive. The system then displays the recommended movies and their similarity scores.



16. Testing:

I tested the recommendation system using different movie titles. I also tested the interface by entering a movie title and checking whether the recommendations were displayed correctly. An additional check was added for invalid movie titles. If a movie title is not found in the dataset, the system displays a message asking the user to enter an exact movie title from the dataset. This helped make the interface easier to use.

17. Challenges and Limitations:

One limitation of this project is that the recommendation system mainly uses movie genres. Because of this, it does not consider other information such as:

- Storyline
- Actors
- Directors
- User's personal taste
- Previous user ratings

Another limitation is that the evaluation method is based on shared genres. A movie can share a genre with another movie but the user may still not like it. The system can be improved in the future by using more movie information and user behavior.

18. Future Improvements:

Some possible improvements for this project are:

- Add movie descriptions.
- Include actors and directors.
- Use user ratings more directly.
- Build a hybrid recommendation system.
- Add collaborative filtering.
- Improve the user interface.
- Allow users to select multiple favorite movies.
- Use more detailed movie features for recommendations.

These improvements could make the recommendations more personalized.

19. Tools and Technologies Used:

The following tools and technologies were used in this project:

- **Python:** Main programming language.
- **Google Colab:** Development environment.
- **Pandas:** Data loading and preprocessing.
- **NumPy:** Numerical operations.
- **Scikit-learn:** TF-IDF and cosine similarity.
- **Gradio:** Demonstration interface.

- **MovieLens:** Movie dataset.

20. Conclusion:

In this project, I developed a basic movie recommendation system using content-based filtering. The MovieLens dataset was cleaned and prepared before building the recommendation engine. Movie genres were converted into numerical features using TF-IDF, and cosine similarity was used to find similar movies. The system was tested with multiple movies and achieved an average Precision@5 of **1.00** according to the genre-based relevance rule used in this project. A simple Gradio interface was also created to make the system easier to demonstrate and use. Overall, this project helped me understand the basic workflow of a recommendation system, including data preparation, feature extraction, similarity calculation, evaluation, and interface development.